INTERNAL DEVELOPER REFERENCE

infraflow
Agentic AI Layer

Implementation record — Milestones M0 & M1
Multi-agent architecture review, live progress, and the LangGraph orchestration behind it

Scope covered: M0 (single grounded Architect agent) & M1 (LangGraph supervisor + parallel specialists, evidence gate, live polling UI)

Codebase: infra-as-a-code / infraflow — React+TS frontend, Node/Express backend ("IAAS backend"), new Python/FastAPI agent service ("services/agents")

Status: Implemented, tested end-to-end against real diagrams and a real LLM, one production concurrency bug found and fixed (see §9)

Purpose of this document: the primary reference for any developer picking up this codebase — what exists, why it's shaped this way, how data flows, and exactly what is *not* built yet

Contents

1. Why this exists & the core architectural rule
2. Service topology
3. M0 — Extract the seam
4. M1 — Multi-agent + live progress
5. The LangGraph graph, node by node
6. The five prompts (supervisor + 4 specialists) and the evidence gate
7. Data models
8. API reference
9. Incident record: the concurrency bug and its fix
10. Security & trust boundary
11. Complete file inventory
12. Running & testing it locally
13. What is deliberately NOT built yet (M2–M5)
14. Glossary

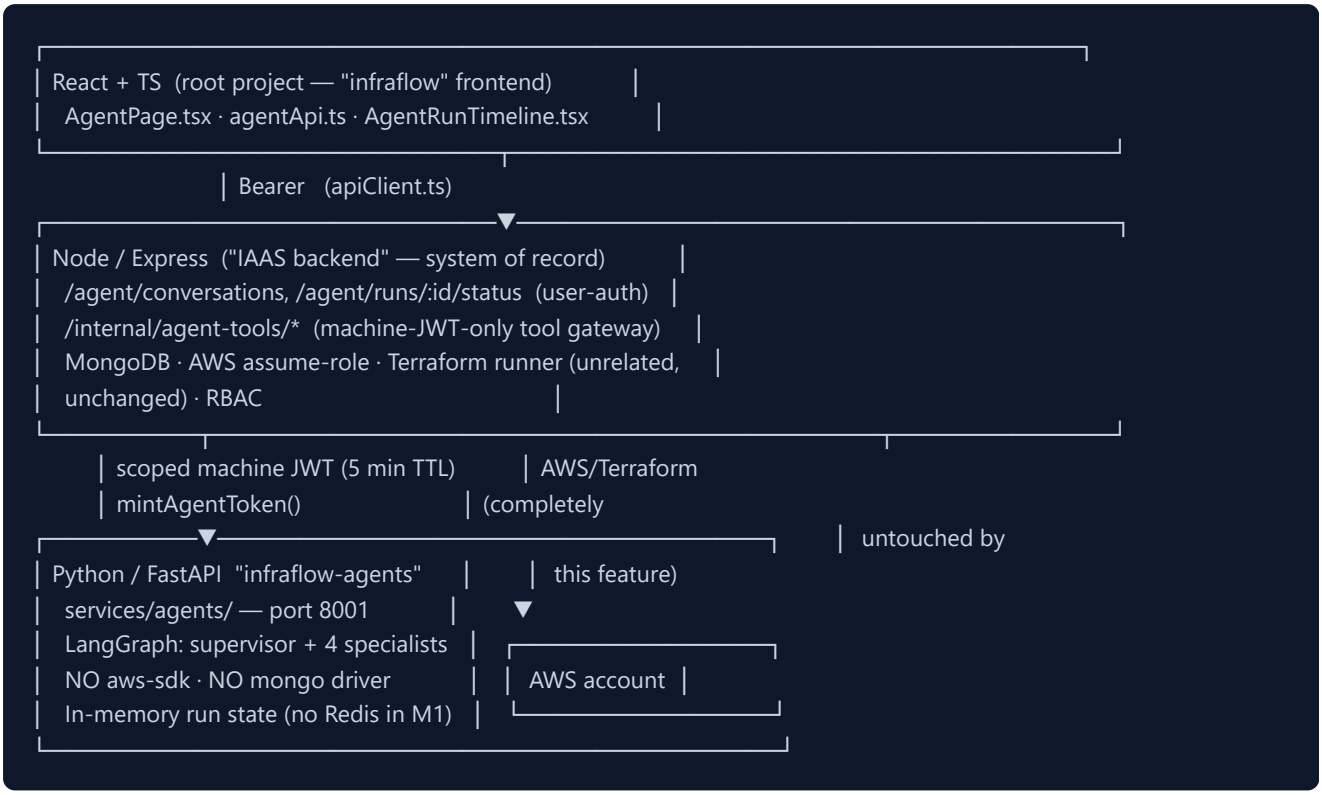
1. Why this exists & the core architectural rule

infraflow's "AI Cloud Agent" page existed before this work but was non-functional — it proxied straight to a RAG FastAPI service (`RAG_API_URL`) that was never actually deployed anywhere in this codebase. This implementation replaces that dead path with a real multi-agent system, built in two milestones (M0, then M1) against a written implementation plan (`infraflow-multi-agent-plan.pdf`).

> **The one rule everything else is built around:** the Python agent service holds the reasoning. The Node/Express backend holds the keys. The agent service has **no AWS SDK and no MongoDB driver** — every capability it has is a call back into Express through an authenticated, scoped "tool gateway." This is enforced as code, not just convention: `services/agents/tests/test_no_cloud_sdk.py` greps the whole service and its `requirements.txt` for `boto3` / `pymongo` / `motor` /any AWS SDK reference and fails if it finds one.

Practically, this means: the agent can read your diagram, your generated Terraform, your validation issues, and your cached AWS billing data. It **cannot** call AWS directly, write to Mongo directly, or trigger a deployment. There is no code path anywhere in this system from an LLM's output to `terraform apply` .

2. Service topology



What runs where

Layer	Location	Port (dev)	Owns
Frontend	`src/` (project root)	5173 (Vite)	Chat UI, live timeline, polling
Backend	`IAAS backend/`	4001	Auth, MongoDB, the tool gateway, run lifecycle records
Agent service	`services/agents/`	8001	LangGraph orchestration, all LLM calls

3. M0 — Extract the seam

M0 M0's goal was narrow on purpose: prove the grounding pipeline and the trust boundary work, with the smallest possible surface area. One agent ("Architect"), no orchestration framework, no streaming — a single OpenAI call per question, grounded in real data.

3.1 The core problem M0 solved

Two real gaps existed in the codebase before this work:

1. **No shared source of truth reachable from the backend.** The frontend's `exportTerraform()`, `validateDiagram()`, etc. (in `src/utils/*.ts`) are pure, portable functions — but the backend is plain Node ESM JavaScript with no TypeScript support, so it couldn't import them. (The backend also has its own, separate, older `terraformGenerator.js` / `diagramValidator.js` used by unrelated existing routes — this work does not touch or reconcile those; it deliberately imports the frontend's canonical `.ts` versions instead of adding a third implementation.)
2. **`AgentPage.tsx` never sent a `diagramId`.** `createAgentConversation()` only ever sent `{title, message}`. So even though `AgentConversation.context.diagram` existed as a schema field, it was always empty in practice.

Fix for gap 1 — `tsx` as a Node ESM loader

Rather than a heavier "extract into a shared npm workspace" refactor, the backend's `package.json` dev/start scripts were changed to register `tsx` as a loader:

```
"dev": "nodemon --exec \"node --import tsx/esm src/server.js\"",
"start": "node --import tsx/esm src/server.js"
```

This makes `.ts` files directly importable from plain `.js` files, at zero build-step cost:

```
// IAAS backend/src/controllers/internalAgentToolsController.js
import { exportTerraform } from '../../../src/utils/exportTerraform.ts';
import { validateDiagram } from '../../../src/utils/validate.ts';
import { validateGeneratedTerraform } from '../../../src/utils/terraformValidation.ts';
import { createDeploymentPlan } from '../../../src/utils/deploymentPlan.ts';
import { diagramStructureKey } from '../../../src/utils/graphIndex.ts';
```

These five functions (and their transitive imports — `resourceRequirements.ts`, `diagramSemantics.ts`, `nodeMetrics.ts`, `awsServices.ts`) were verified to be pure — no runtime React/DOM dependency, only `import type` references to `reactflow`, which are erased at compile time.

Fix for gap 2 — server-side diagram fallback

Since the frontend contract couldn't be changed in M0, `agentBridge.js` resolves a missing `diagramId` by falling back to the workspace's most-recently-updated `Diagram`:

```
async function mostRecentDiagramId(workspaceId) {
  const diagram = await Diagram.findOne({ workspace: workspaceId })
```

```
    .sort({ updatedAt: -1 }).select('_id').lean();
  return diagram?._id?.toString();
}
```

3.2 The M0 tool gateway

`POST /internal/agent-tools/grounding` is the one endpoint M0 introduced. It is the single batched call an agent run makes to learn everything about a diagram — nodes, edges, generated Terraform, validation issues, and a deployment plan — computed fresh from the same functions the browser itself calls:

```
// getGrounding (internalAgentToolsController.js)
const { nodes, edges, activeRegion } = diagram;
const terraform = exportTerraform(nodes, edges);
const validation = [...validateDiagram(nodes, edges, activeRegion),
                    ...validateGeneratedTerraform(terraform)];
const plan = createDeploymentPlan(nodes, edges, validation);
// → { diagramId, diagramName, nodes, edges, activeRegion, digest, terraform, validation, plan,
awsInsights }
```

3.3 M0's request flow (superseded — see §4)

> M0 originally used a *synchronous* flow: `agentController.js` awaited a single OpenAI call end-to-end and returned the finished answer in one HTTP response, via a since-removed `POST /runs/message` endpoint on the Python side. M1 replaced this entirely with the asynchronous run lifecycle described in §4 — `/runs/message` no longer exists in the code. It's recorded here only because M0's grounding endpoint and diagram-fallback logic are still exactly as originally built and remain load-bearing for M1.

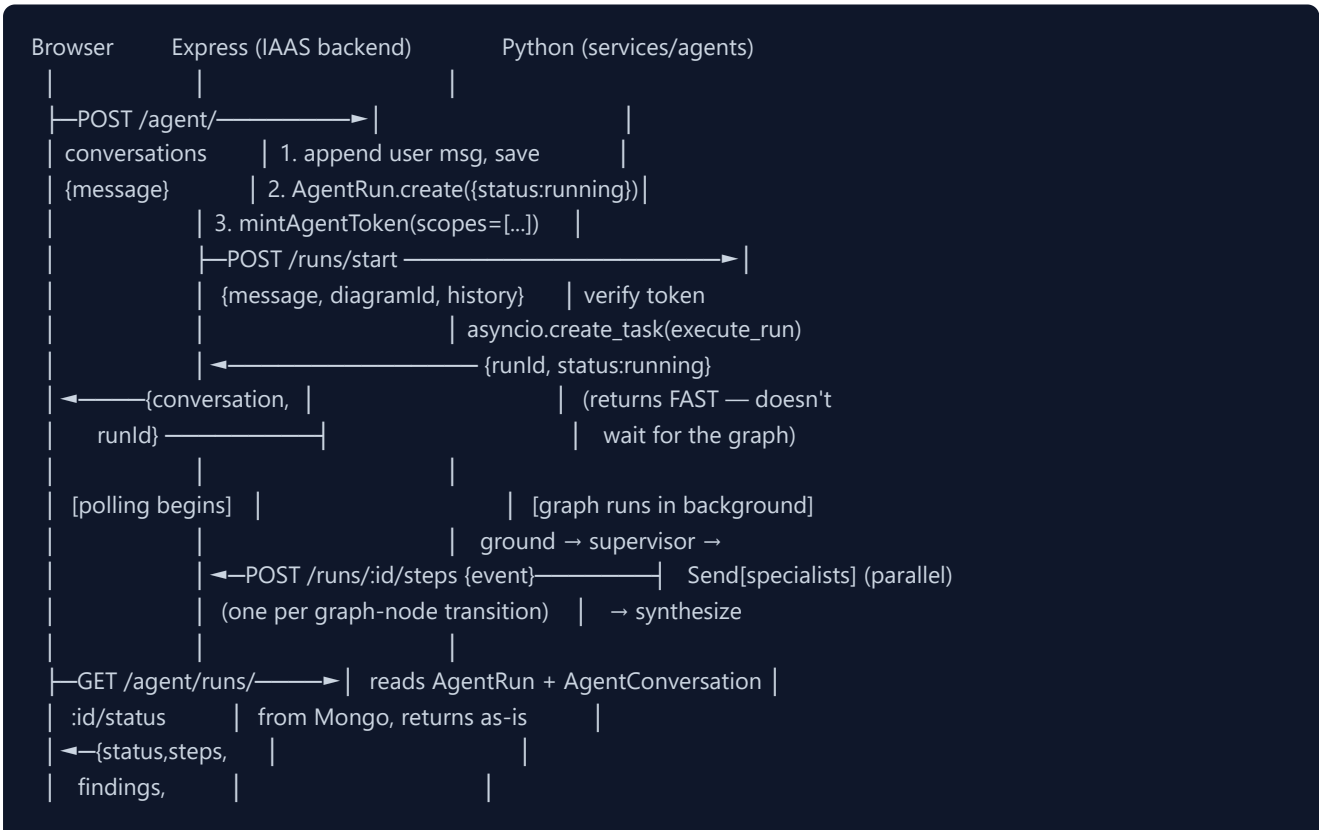
4. M1 — Multi-agent + live progress

M1 M1 replaced the single Architect agent with a real LangGraph supervisor routing to up to four parallel specialists (Architect, Terraform, Cost, Security), added a mechanical evidence gate, and made the whole run observable live in the UI — without introducing a new "live updates" pattern into the app.

4.1 Two decisions made before building, and why

Decision	Reasoning
Polling, not SSE. `GET /agent/runs/:id/status` is polled every 1.5s from the browser, exactly like `DeploymentLiveMonitor.tsx` already polls deployment status (2.5s) and GitHub run status (8s).	This codebase has *zero* existing SSE/EventSource infrastructure. Every other "live" feature in the app already polls, and `app.js` already carves out a separate, more generous rate limit specifically for polling routes (`POLLING_ROUTE_PATTERNS` / `POLLING_RATE_LIMIT_MAX`). Reusing a proven pattern beats introducing a new one, especially since `EventSource` can't set an `Authorization` header anyway.
Real LangGraph, not a hand-rolled asyncio orchestrator.	LangGraph's `Send` API is exactly the "supervisor routes to N of M possible specialists in parallel" primitive this needs. It does **not** require Redis for M1 — the checkpointer/ `interrupt()` features that actually need Redis are an M2 concern (pausing a run for human approval), not used here. The graph runs fully in-memory via `graph.astream(...)` .

4.2 The run lifecycle, end to end



```
│  conversation}──────────┤                          │
│ (repeat every 1.5s) │                          │
│                  │ ◄─POST /runs/:id/complete──────────────┤ (after graph finishes)
│                  │  {status, content, metadata}      │  retries 3× if this fails
│                  │  → pushes assistant message,      │
│                  │    sets AgentRun.status='completed' │
│ ◄─ poll returns status:'completed' + the new message ──────────┤
│  timeline settles, chat shows the answer                  │
```

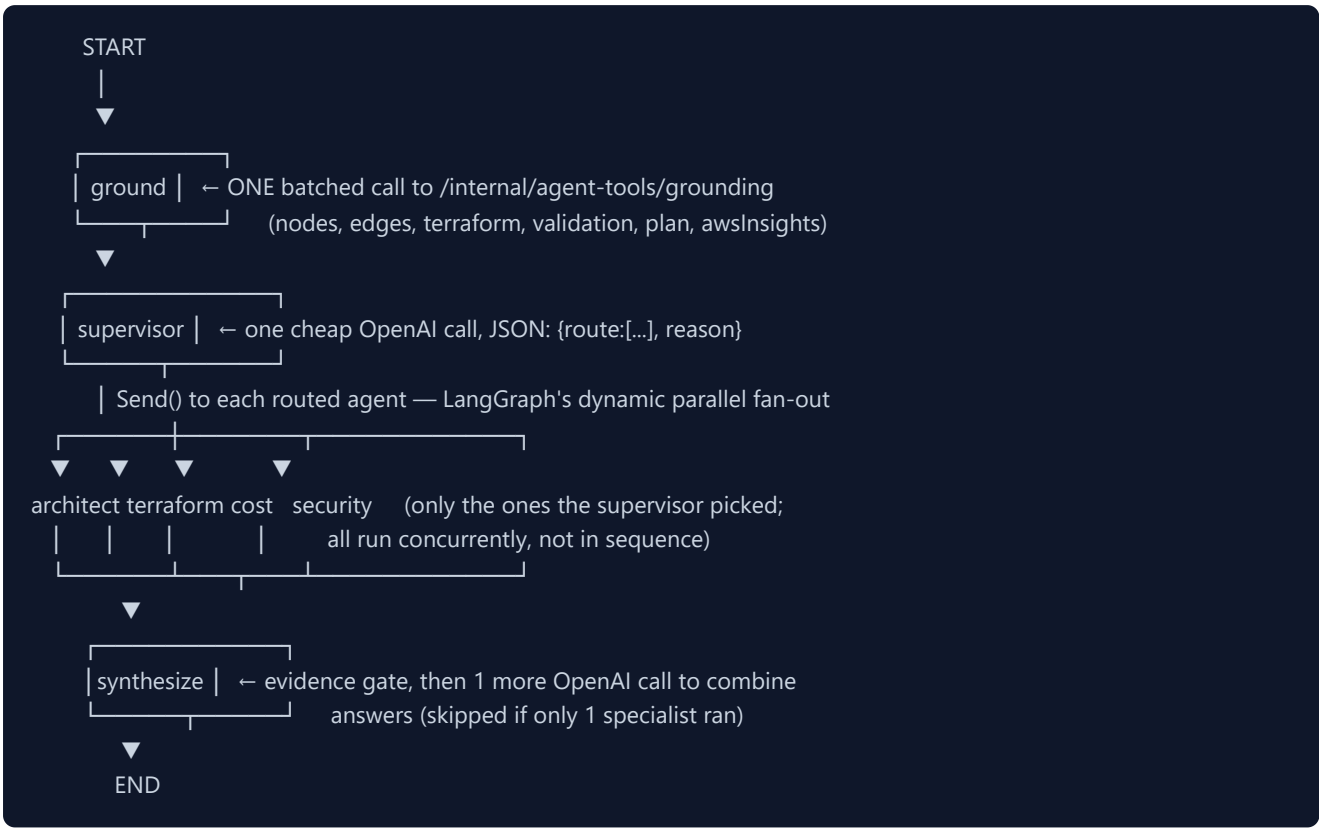
4.3 Frontend: `AgentPage.tsx` + `AgentRunTimeline.tsx`

`AgentPage.tsx` tracks an `activeRunId`. While it's set, a `useEffect` polls `getAgentRunStatus(runId)` every `RUN_POLL_MS` (1500ms) and feeds the result into two places: `setRunStatus` (drives the timeline) and `setConversation` (so the chat always reflects Mongo's current state, including once the assistant message lands). Polling stops the instant `status !== 'running'`.

A client-side backstop (`MAX_POLL_MS = 90_000`) guarantees the UI can never spin forever even under an unforeseen failure — it surfaces "this run may be stuck" instead.

`AgentRunTimeline.tsx` renders one row per routed agent (colour-coded to match `FlowEdge.tsx`'s existing edge-category palette — security is red, terraform green, architect purple, cost amber), using the exact same status-glyph vocabulary (`Loader2` / `CheckCircle2` / `Circle` / `AlertTriangle`) as `DeploymentLiveMonitor.tsx`.

5. The LangGraph graph, node by node



Defined in `NEW` `services/agents/app/graph.py`. Every node is a plain async function taking/returning a slice of `RunState` (a `TypedDict`); LangGraph merges parallel writes to `results` / `findings` via an `Annotated[list[...], operator.add]` reducer, so four specialists writing concurrently can never clobber each other.

Node	Function	Does
`ground`	`ground_node`	Calls `fetch_grounding()`, stores the raw bundle dict + a compact DSL projection (`build_dsl()`, `app/dsl.py`) into state.
`supervisor`	`supervisor_node`	Builds a one-line diagram summary, calls `supervisor.route()`. Falls back to `["architect"]` on any parse failure — routing can degrade, never disappear.
`architect` / `terraform` / `security`	`*_node`	Call the matching specialist's `run()`, which delegates to the shared `run_specialist()` helper (one JSON-mode LLM call → `{content, findings}`).
`cost`	`cost_node`	Same shape, but also extracts `aws_insights` from the grounding bundle and appends an `<aws_insights>` block to the prompt.
`synthesize`	`synthesize_node`	Runs `apply_evidence_gate()` over every collected `Finding`, then `synthesize.synthesize()` to produce the final chat-readable answer.

5.1 `app/runner.py` — turning graph execution into live progress

Rather than threading "report progress" logic into every node function, `runner.py` observes the graph from outside using `graph.astream(state, stream_mode="updates")`, which yields one update per completed node. Each update is translated into a step callback POSTed to Express:

```
async for update in _graph.astream(initial_state, stream_mode="updates"):
    for node_name, delta in update.items():
        if node_name == "ground": → {"type": "grounding.ready"}
        elif node_name == "supervisor": → {"type": "route.decided", "route": [...], "reason":
...}
        elif node_name in SPECIALIST_NODES: → {"type": "agent.finished", "agent": ...,
"summary": ..., "findingsCount": ...}
        elif node_name == "synthesize": → final_content, final_metadata captured for the
/complete callback
```

The whole thing is wrapped in `asyncio.wait_for(..., timeout=MAX_WALL_SECONDS)` (60s) — a lightweight version of the plan's `RunBudget`. A timeout posts a `status:'failed'` completion with a friendly message rather than leaving the run stuck.

The background task itself is fire-and-forget from `POST /runs/start`'s point of view, but its `asyncio.Task` reference is kept alive in a module-level `set()` (discarded via `add_done_callback` on completion) — a well-known asyncio footgun otherwise: a task nothing awaits can be garbage-collected mid-run.

6. The five prompts and the evidence gate

6.1 Supervisor (`app/agents/supervisor.py`)

One `response_format: json_object` call. Given the question and a one-line diagram summary (resource count, region, error/warning counts), returns `{"route": [...], "reason": "..."}`. A broad question ("review this architecture") should route to several agents; a narrow one ("what's my riskiest resource") should route to one or two. Verified live: a cross-domain question correctly routed to all four agents in one run.

6.2 The shared specialist contract (`app/agents/base.py`)

All four domain agents share one code path, `run_specialist()`, so the JSON-parsing/error-handling logic isn't duplicated four times. Every specialist's system prompt is suffixed with the same `FINDING_JSON_INSTRUCTIONS`, which demands a single JSON object back:

```
{
  "content": "<1-4 sentence chat-readable answer>",
  "findings": [
    { "severity": "info"|"warning"|"error", "title": "...", "detail": "...",
      "evidence": [{"kind": "node"|"terraform_block", "ref": ""}] }
  ]
}
```

A specialist's failure (bad JSON, API error, timeout) is caught inside `run_specialist()` and turned into a graceful fallback `SpecialistResult` — one specialist erroring can never sink the whole parallel fan-out.

Agent	File	Domain	Extra context beyond the diagram DSL
Architect	`architect.py`	Overall structure/topology	—
Terraform	`terraform.py`	Generated HCL correctness/completeness	—
Cost	`cost.py`	Spend, waste, rightsizing	`<aws_insights>` block (month-to-date total + top services), or an explicit "no AWS account connected" note
Security	`security.py`	Public exposure, encryption, IAM-adjacent config *visible in the diagram*	Explicitly told it has **no live IAM data** this run — see §13

6.3 Synthesize (`app/agents/synthesize.py`)

If only one agent ran, its `content` is used directly — no extra LLM call. Otherwise, one more call combines every specialist's answer into 1–2 coherent paragraphs, explicitly instructed to preserve every concrete fact (node ids, dollar figures, Terraform addresses) rather than smoothing them away. On failure, falls back to the raw concatenated specialist answers rather than erroring.

6.4 The evidence gate (`app/evidence.py`)

Mechanical, not prompt-based — this is the anti-hallucination mechanism. A `Finding` survives only if every one of its `evidence[]` entries points at something that actually exists in that run's grounding bundle:

```
BLOCK_RE = re.compile(r'^\s*(resource|data)\s+"([^"]+)"\s+"([^"]+)"\s*\{')  # mirrors
terraformValidation.ts

def verify_evidence(finding, bundle):
    node_ids = {n.get("id") for n in bundle.nodes}
    tf_addresses = declared_resource_addresses(bundle.terraform)
    for ev in finding.evidence:
        ref = _normalize_ref(ev.ref)             # strips [brackets] — see §9
        if ev.kind == "node" and ref not in node_ids: return False
        if ev.kind == "terraform_block" and ref not in tf_addresses: return False
    return True
```

Verified live: an unfiltered, evidence-free run produced 0 surviving findings out of 12 raised (the bug in §9); after the fix, the same class of question reliably produces real, node-cited findings (9 survivors out of the raised set in one recorded test run).

7. Data models

7.1 `AgentRun` — NEW `IAAS backend/src/models/AgentRun.js`

The single new persistence model this feature introduced. Mongo's own `_id` doubles as the `runId` embedded in the machine JWT — there is no separate id scheme.

Field	Type	Notes
`workspace`	ObjectId → Workspace	Every query is scoped by this — see §10
`conversation`	ObjectId → AgentConversation	Where the final assistant message gets pushed
`user`	ObjectId → User	Who started the run
`status`	'running'|'completed'|'failed'	What the browser polls for
`route`	string[]	Which agents the supervisor picked
`steps`	Mixed[]	Append-only log of every callback Python posted (see §4.2)
`findings`	Mixed[]	The evidence-gate-surviving `Finding` objects, set once on completion

7.2 Python `RunState` — `services/agents/app/state.py`

A `TypedDict(total=False)` — LangGraph's state object for one run. Not persisted anywhere; lives only for the duration of the background task.

```
class RunState(TypedDict, total=False):
    run_id: str; diagram_id: str; question: str; history: list[dict]
    bearer_token: str            # the SAME machine JWT Express minted — echoed on every
callback
    grounding: dict               # raw GroundingBundle dict from Express
    dsl: str                      # compact projection, see §5
    route: list[AgentName]; route_reason: str
    results: Annotated[list[SpecialistResult], add]   # parallel-safe accumulation
    findings: Annotated[list[Finding], add]           # parallel-safe accumulation
    final_findings: list[Finding]  # evidence-gate output — written once, no reducer
    final_content: str
```

7.3 `Evidence` / `Finding` / `SpecialistResult` (pydantic, `state.py`)

```
class Evidence(BaseModel):
    kind: Literal["node", "terraform_block"]
    ref: str                       # a node id, or a `type.name` Terraform address

class Finding(BaseModel):
    agent: AgentName
    severity: Literal["info", "warning", "error"]
```

```
    title: str; detail: str
    evidence: list[Evidence] = []

class SpecialistResult(BaseModel):
    agent: AgentName; content: str; findings: list[Finding] = []
```

7.4 `GroundingBundle` (Python, `app/gateway.py`)

A thin wrapper around the JSON Express's `/grounding` endpoint returns — `.raw` keeps the original dict so it round-trips cleanly through LangGraph state without a second fetch. Fields: `diagram_id`, `diagram_name`, `nodes`, `edges`, `active_region`, `digest`, `terraform`, `validation`, `plan`, `aws_insights`.

8. API reference

8.1 Public endpoints (user session JWT, `requireAuth` + `canUseAiAgent` — Super Admin only)

Method & path	Body	Returns
`GET /api/v1/agent/conversations`	—	`AgentConversation[]`
`POST /api/v1/agent/conversations`	`{title?, diagramId?, awsAccountId?, message?}`	`{conversation, runId?}` — `runId` present only if `message` was included
`POST /api/v1/agent/conversations/:id/messages`	`{message}`	`{conversation, runId}`
`GET /api/v1/agent/runs/:id/status`	—	`{status, route, steps, findings, conversation}`

Note the deliberate shape change from before this work: the assistant's reply is **not** in the response to the two POST endpoints anymore — it only appears once polling reaches a terminal `status`. This is why `AgentPage.tsx` needed changes in M1 (it didn't in M0).

8.2 Internal tool gateway (machine JWT only, `requireAgentScope` — never `requireAuth`)

Method & path	Required scope	Body	Returns
`POST /api/v1/internal/agent-tools/grounding`	`diagram:read`	`{diagramId, include?: string[]}`	Full grounding bundle (§3.2) + `awsInsights` if `include` asked for it *and* the token carries `aws:read:billing`
`POST /api/v1/internal/agent-tools/runs/:runId/steps`	any valid agent token, `runId` must match the token's own `run` claim	`{event}`	`{success:true}`
`POST /api/v1/internal/agent-tools/runs/:runId/complete`	same as above	`{status, content, metadata}`	`{success:true}`

8.3 Python service (FastAPI, port 8001 — only ever called by Express)

Method & path	Auth	Behaviour
`GET /health`	none	`{ok:true, service:"infraflow-agents"}`
`POST /runs/start`	Bearer machine JWT, requires `diagram:read`	Schedules the LangGraph run as a background task, returns `{runId, status:"running"}` **immediately** — never waits for the graph to finish

9. Incident record: the concurrency bug and its fix

> **Symptom (reported by a developer testing the feature):** the UI got stuck showing "Synthesizing answer" indefinitely, even though the underlying LLM calls had genuinely completed.

9.1 Root cause

`postRunStep` and `postRunComplete` (in `internalAgentToolsController.js`) originally did `AgentRun.findOne()` → mutate the in-memory document → `.save()`. Up to four specialists finish within a fraction of a second of each other (plus the terminal `synthesize` callback right after) — several concurrent HTTP requests land on the *same* Mongo document at nearly the same instant. This is Mongoose's well-known optimistic-concurrency race: a losing `.save()` throws a `VersionError`.

Compounding it: `services/agents/app/gateway.py`'s `post_run_step` / `post_run_complete` never checked the HTTP response status (no `raise_for_status()`). A 500 from Express was silently treated as success from Python's point of view. Net effect: the OpenAI call genuinely finished, but the write that should have flipped `AgentRun.status` to `'completed'` sometimes never landed — leaving the frontend polling a run stuck at `'running'` forever, with no error anywhere to point at.

9.2 The fix — three layers

Layer	Change
Express (root cause)	Both handlers switched to atomic `findOneAndUpdate` with `$push` / `$set` — no read-modify-write, so there is nothing left to race on.
Python (defense in depth)	`raise_for_status()` added to both gateway calls; `post_run_complete` specifically retries 3× with backoff, since it is the single terminal callback — losing it strands the run with no recovery path.
Frontend (backstop)	`AgentPage.tsx` gained a 90-second client-side polling deadline, so *any* unforeseen failure mode surfaces an error instead of spinning forever.

```
// AFTER — internalAgentToolsController.js
export const postRunStep = asyncHandler(async (req, res) => {
  const update = { $push: { steps: { ...event, at: new Date() } } };
  if (event.type === 'route.decided') update.$set = { route: event.route };
  const run = await AgentRun.findOneAndUpdate({ _id: req.params.runId, workspace:
req.agent.workspaceId }, update);
  ...
});
```

9.3 A related, smaller bug found and fixed the same session

Before the fix above, a full end-to-end test surfaced a *second* bug: 0 of 12 raised findings survived the evidence gate. The LLM was copying the literal `[brackets]` from the prompt's own example text into `evidence.ref` — `"[tpl-starter-ec2]"` instead of `"tpl-starter-ec2"` — so the ref never matched a real

node id. Fixed both ways: the prompt was reworded to be unambiguous, **and** `evidence.py`'s `_normalize_ref()` now strips `[]`/whitespace defensively before comparing — because, per the plan's own philosophy, prompt discipline alone is never trustworthy; the mechanical check has to be lenient to real-world model output.

9.4 Verification performed

After both fixes, the exact cross-domain question that exercises the race (fans out to all four agents) was run **five times back-to-back** through the real API. All five reached `completed` cleanly with the assistant reply present — 5/5, where before the fix this was a genuine, timing-dependent intermittent failure.

10. Security & trust boundary

10.1 The machine JWT

Minted once per run by `agentIdentity.js`'s `mintAgentToken()`, signed with `AGENT_TOOL_SECRET` (HS256), 5-minute expiry:

```
{ sub: "agent:", aud: "agent-tools", act: "",
  workspace: "", scope: [...], run: "" }
```

Python's `verify_agent_token()` (`app/security.py`) verifies this same token and **never mints its own** — it just forwards the identical token on every callback into Express. This is deliberate: it means the token that authorized reading a diagram is the exact same token that gets to report progress and post the final answer, and Express's `requireAgentScope()` middleware is the single place any of it is enforced.

10.2 Scopes actually used in M1

One token is minted per run with the union of everything any M1 agent might need — a simplification versus the original plan's per-agent `SCOPES_BY_AGENT` table (worth revisiting once patch-proposal tools arrive in M2 and the blast radius of a leaked/misused token starts to matter more):

```
const RUN_SCOPES = ['diagram:read', 'terraform:generate', 'aws:read:billing'];
```

The grounding endpoint double-gates AWS billing data: it's only attached if the caller both *asked* for it (`include`) *and* the token actually *carries* `aws:read:billing`.

10.3 Workspace isolation

Every Mongo query on the agent side is filtered by `req.agent.workspaceId` (tool-gateway routes) or `req.user.workspace` (user-facing routes) — the agent service itself never constructs a query; it only ever passes an id, so a compromised or confused run cannot express a cross-workspace query even in principle.

10.4 `assertOwnRun`

Both `/runs/:runId/steps` and `/runs/:runId/complete` check `req.agent.runId === req.params.runId` — a token minted for run A can never report progress on run B, even though all agent tokens share the same secret.

10.5 What the agent literally cannot do

There is no privileged tool bound anywhere in this Python service. `apply` / `destroy` /any AWS-mutating action isn't a registered tool, isn't a scope, and isn't reachable — the service has no AWS SDK installed at all (enforced by `tests/test_no_cloud_sdk.py`). The agent can produce text and structured findings. It cannot deploy anything.

11. Complete file inventory

11.1 Frontend (project root)

scripts/gen-agent-schemas.ts	NEW	M0 — awsServices.ts → aws_services.json
src/dashboard/agentApi.ts	MOD	M1 — run/status types, getAgentRunStatus()
src/dashboard/pages/AgentPage.tsx	MOD	M1 — polling, timeline, 90s backstop
src/dashboard/components/AgentRunTimeline.tsx	NEW	M1 — live progress UI
src/dashboard/dashboardData.ts	MOD	M1 — 2 cross-domain agentActions prompts
src/styles.css	MOD	M1 — .agent-timeline* block
package.json	MOD	M0 — tsx devDependency, gen:agent-schemas

11.2 Express backend (`IAAS backend/`)

package.json	MOD	M0 — tsx dep, --import tsx/esm loader
.env.example	MOD	M0 — AGENT_API_URL, AGENT_TOOL_SECRET
src/config/env.js	MOD	M0 — reads the two vars above
src/services/agentIdentity.js	NEW	M0 — mintAgentToken()
src/middleware/requireAgentScope.js	NEW	M0 — verifies machine JWT + scopes
src/models/AgentRun.js	NEW	M1 — run status/steps/findings record
src/utils/awsInsightsForAgent.js	NEW	M1 — cached billing lookup for Cost agent
src/controllers/internalAgentToolsController.js	NEW	M0, FIX M1 — grounding + run callbacks
src/routes/internalAgentTools.js	NEW	M0, MOD M1 — added the two run routes
src/routes/index.js	MOD	M0 — mounts /internal/agent-tools
src/utils/agentBridge.js	NEW	M0 (as answerWithAgent), rewritten M1 (startAgentRun)
src/controllers/agentController.js	MOD	M0 & M1 — async launchRun(), getRunStatus
src/routes/agentRoutes.js	MOD	M1 — added GET /runs/:id/status
src/utils/agentResponder.js		(pre-existing — superseded, left in place, unused)

11.3 Python agent service (`services/agents/` — entirely new)

requirements.txt	fastapi, uvicorn, pydantic(-settings), httpx, openai, python-jose, python-dotenv, langgraph, pytest
.env.example / .env	OPENAI_API_KEY, OPENAI_AGENT_MODEL, AGENT_TOOL_SECRET, EXPRESS_INTERNAL_URL
app/config.py	Settings (pydantic-settings)
app/security.py	AgentIdentity, verify_agent_token()
app/gateway.py	GroundingBundle, fetch_grounding, post_run_step, post_run_complete **FIX** raise_for_status + retry
app/dsl.py	build_dsl() — compact line-oriented diagram projection
app/state.py	**NEW** M1 — Evidence, Finding, SpecialistResult, RunState
app/evidence.py	**NEW** M1, **FIX** — the evidence gate
app/agents/base.py	**NEW** M1 — shared LLM-call + JSON-contract helper
app/agents/architect.py	M0 (single-shot) **rewritten** M1 onto base.py
app/agents/terraform.py	**NEW** M1
app/agents/cost.py	**NEW** M1 — + AWS insights block
app/agents/security.py	**NEW** M1
app/agents/supervisor.py	**NEW** M1 — routing
app/agents/synthesize.py	**NEW** M1 — final answer combination
app/graph.py	**NEW** M1 — the LangGraph StateGraph (§5)
app/runner.py	**NEW** M1 — background execution + progress reporting
app/api/routes_runs.py	M0 (/runs/message) **replaced** M1 (/runs/start)
app/main.py	FastAPI app + /health
app/tools/schemas/aws_services.json	GENERATED — do not hand-edit (see §13)
tests/test_no_cloud_sdk.py	M0 — the architectural guarantee, enforced as a test
README.md	M0 — local setup instructions

12. Running & testing it locally

12.1 Prerequisites

- MongoDB running locally (this dev environment uses it as a Windows service already).
- Node 22+, with an `OPENAI_API_KEY` available (this project reads it either from `services/agents/.env` or, if unset there, from a Windows user environment variable — either works).
- Python 3.12 or 3.13 recommended for the agent service's venv (3.14 is new enough that some pinned wheels may lack prebuilt binaries).

12.2 Start order

```
# 1. Python agent service
cd services/agents
py -3.12 -m venv .venv
.venv\Scripts\pip install -r requirements.txt
copy .env.example .env    # then fill in OPENAI_API_KEY + AGENT_TOOL_SECRET
.venv\Scripts\uvicorn app.main:app --port 8001 --reload

# 2. Express backend  (separate terminal)
cd "IAAS backend"
npm install
npm run dev              # boots on :4001, uses --import tsx/esm automatically

# 3. Frontend  (separate terminal)
npm install               # project root
npm run dev               # Vite, :5173
```

> **AGENT_TOOL_SECRET must be identical** in `IAAS backend/.env` and `services/agents/.env` — it's the shared secret both sides use to sign/verify the machine JWT. A mismatch surfaces as every internal-tool call returning 401.

12.3 Exercising it

1. Log in as a user with `role: 'superadmin'` (the page is gated by `canUseAiAgent`).
2. Open a diagram in the Visual Builder first, so a `Diagram` document exists (grounding falls back to the workspace's most-recently-updated one if the conversation doesn't have one attached).
3. Go to **AI Cloud Agent** in the dashboard sidebar.
4. Try a broad question ("Review this architecture") and a narrow one ("What's my riskiest resource?") — the supervisor should route differently each time; watch the timeline populate live.

12.4 Automated checks

```
cd services/agents && .venv\Scripts\pytest             # architectural guard (§1) + evidence-gate unit checks
```

```
cd "IAAS backend" && npm run dev                              # boots clean if tsx wiring is intact
cd (project root) && npx tsc --noEmit                          # frontend type-checks
```

13. What is deliberately NOT built yet (M2–M5)

Everything below is real, scoped-out work from the source plan (`infraflow-multi-agent-plan.pdf`) — not an oversight. Listed here so nobody re-discovers these as "missing" without knowing they were a conscious sequencing decision.

Not built	Where it belongs	Why it's not in M0/M1
Diagram patch proposals / `ChangePlan`	M2	Needs the closed-vocabulary validation the generated `aws_services.json` already exists for but nothing consumes yet.
Human-in-the-loop approval, LangGraph `interrupt()`	M2	Requires a Redis checkpointer to pause/resume a run across process boundaries — not needed while every M1 run completes in one shot.
Blast-radius classification, tiered auto/review/elevated/blocked gating	M3	Only meaningful once there's something to gate (patch proposals from M2).
Drift agent (reality vs. intent)	M3	Needs the deployment-drift plumbing, a separate concern from diagram-time review.
Live IAM role reads bound to the Security agent	Reasonable M2+ addition	`GET /aws/accounts/:id/iam-roles` always hits AWS live with no caching — too slow/fragile to bind into a 4-way parallel fan-out today. Security currently reasons only over diagram-visible config (security groups, IAM nodes) and validator output.
`search_well_architected` RAG tool	Deferred indefinitely	No RAG FastAPI service exists anywhere in this codebase or a sibling directory — `RAG_API_URL` / `agentResponder.js` are dead legacy code, not wired to anything.
Per-agent scoped tokens (vs. one run-wide token)	Worth revisiting alongside M2	Simplification made explicit in §10.2 — matters more once mutating tools exist.
Langfuse/OTel tracing, `AgentRun`-based observability dashboard	M4	Not needed to prove the core mechanism works; `AgentRun.steps` already gives basic replay today.
Formal eval suite (contract / trajectory / scenario / adversarial)	M4	Only `test_no_cloud_sdk.py` exists today — a real regression harness is future work.
The no-cloud-SDK guard wired into CI	Any time	Runs locally via `pytest` today; no GitHub Actions workflow currently touches this service.

14. Glossary

Term	Meaning in this codebase
Grounding bundle	The single batched payload (nodes, edges, Terraform, validation, AWS insights) an agent run fetches once and every specialist reads from — never re-fetched per agent.
DSL (diagram DSL)	The compact, line-oriented text projection of a diagram (`build_dsl()`) that agents actually see in their prompts — `[node-id] serviceId "label" key=value` lines plus edges and a validation summary.
Evidence gate	The mechanical check (`app/evidence.py`) that drops any `Finding` citing a node id or Terraform address that doesn't really exist in the grounding bundle.
Machine JWT	The short-lived, scoped token Express mints per run (not the user's own session token) that authenticates every call in both directions between Express and the Python service.
Tool gateway	The `/internal/agent-tools/*` routes — the only way the Python service can read anything from MongoDB; it holds no direct database connection.
Run	One multi-agent invocation, backed by one `AgentRun` Mongo document, tracked by `runId` from the moment a message is sent until `status` reaches a terminal value.
Specialist	One of the four domain agents (Architect, Terraform, Cost, Security) the supervisor can route a question to.